Pipeline

Pipeline - Introduction

- A pipeline in machine learning is a structured sequence of data processing and modeling steps that are chained together to automate the end-to-end machine learning workflow — from raw data input to final prediction.

Pipeline - Introduction

- A pipeline in machine learning is a structured sequence of data processing and modeling steps that are chained together to automate the end-to-end machine learning workflow — from raw data input to final prediction.

- It typically includes steps such as:
  - Data Ingestion (to load data from various sources),
  - Data Cleaning (to handle missing values, duplicates, and inconsistencies),
  - Data Preprocessing (to scale, encode, or normalize data),
  - Feature Engineering (to create or transform features for better learning),
  - Feature Selection or Dimensionality Reduction (to reduce noise and improve model performance),
  - Model Training (to fit an algorithm to the training data),
  - Model Evaluation (to assess performance using metrics and validation),
  - Hyperparameter Tuning (to optimize model and transformation parameters), and finally,
  - Prediction (or Inference) (to apply the trained model to new, unseen data).

Pipeline - Introduction

- A pipeline in machine learning is a structured sequence of data processing and modeling steps that are chained together to automate the end-to-end machine learning workflow — from raw data input to final prediction.

- It typically includes steps such as:
  - Data Ingestion (to load data from various sources),
  - Data Cleaning (to handle missing values, duplicates, and inconsistencies),
  - Data Preprocessing (to scale, encode, or normalize data),
  - Feature Engineering (to create or transform features for better learning),
  - Feature Selection or Dimensionality Reduction (to reduce noise and improve model performance),
  - Model Training (to fit an algorithm to the training data),
  - Model Evaluation (to assess performance using metrics and validation),
  - Hyperparameter Tuning (to optimize model and transformation parameters), and finally,
  - Prediction (or Inference) (to apply the trained model to new, unseen data).

- The pipeline ensures that the same sequence of transformations is consistently applied during both training and inference, thereby reducing errors, preventing data leakage, and increasing efficiency.

Pipeline - sklearn

- class sklearn.pipeline.Pipeline(steps, …) (steps is a list of tuples. List of (each tuple is name of step, estimator) tuples that are to be chained in sequential order.)

Source: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Pipeline - sklearn

- class sklearn.pipeline.Pipeline(steps, …) (steps is a list of tuples. List of (each tuple is name of step, estimator) tuples that are to be chained in sequential order.)
- Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.

Source: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Pipeline - sklearn

- class sklearn.pipeline.Pipeline(steps, …) (steps is a list of tuples. List of (each tuple is name of step, estimator) tuples that are to be chained in sequential order.)
- Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.
- Intermediate steps of the pipeline must be transformers, that is, they must implement fit and transform methods. The final estimator only needs to implement fit.

Source: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Pipeline - sklearn

- class sklearn.pipeline.Pipeline(steps, …) (steps is a list of tuples. List of (each tuple is name of step, estimator) tuples that are to be chained in sequential order.)
- Pipeline allows you to sequentially apply a list of transformers to preprocess the data and, if desired, conclude the sequence with a final predictor for predictive modeling.
- Intermediate steps of the pipeline must be transformers, that is, they must implement fit and transform methods. The final estimator only needs to implement fit.
- The purpose of a pipeline is to combine multiple processing and modeling steps into a single workflow that can be cross-validated and optimized together. You can also replace an entire step by assigning a new estimator to its name, or skip a step by setting it to 'passthrough' or None.

Source: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Pipeline - sklearn

sklearn_pipeline.py

```
import pandas as pd
import pickle
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
import numpy as np

# Load the dataset
df = pd.read_csv("Iris.csv")

# Drop the ID column if it exists
df.drop(columns=['Id'], errors='ignore', inplace=True)

# Split features and labels
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

Pipeline - sklearn

```
# sklearn_pipeline.py

# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create a pipeline: scaling + Naive Bayes
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', GaussianNB())
])

# Train the pipeline
pipeline.fit(X_train, y_train)

# Evaluate the pipeline
y_pred = pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Pipeline Model Accuracy: {accuracy:.2f}")
```

Pipeline - sklearn

sklearn_pipeline.py

```
# Save the pipeline
with open("pipeline.pkl", "wb") as f:
    pickle.dump(pipeline, f)

print("Pipeline saved successfully as 'pipeline.pkl'.")

# Load the saved pipeline
with open("pipeline.pkl", "rb") as f:
    pipeline = pickle.load(f)

# Example input for prediction (same number of features as the training data: 4)
# Let's say this is a new Iris flower data point:
X_new = np.array([[5.1, 3.5, 1.4, 0.2]])

# Make prediction
prediction = pipeline.predict(X_new)

# Print result
print(f"Predicted class: {prediction[0]}")
```

Alternatively:

```
# Import necessary library
import joblib

# Save the object
joblib.dump(pipeline, 'pipeline.pkl')

# Load the object
pipeline = joblib.load('pipeline.pkl')
```

Pipeline – sklearn with Hyper-Parameter Tuning

sklearn_gridsearch_pipeline.py

```
import pandas as pd
import pickle
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score
import numpy as np

# Load dataset
df = pd.read_csv("Iris.csv")
df.drop(columns=['Id'], errors='ignore', inplace=True)

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Pipeline – sklearn with Hyper-Parameter Tuning

sklearn_gridsearch_pipeline.py

```
# Pipeline: scaling + classifier
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', GaussianNB())
])

# Define hyperparameter grid
param_grid = {
    'clf__var_smoothing': [1e-9, 1e-8, 1e-7, 1e-6]  # GaussianNB smoothing parameter
}

# Grid search with cross-validation
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

Pipeline – sklearn with Hyper-Parameter Tuning

sklearn_gridsearch_pipeline.py

```
# Best parameters and accuracy
print("Best Parameters:", grid_search.best_params_)
y_pred = grid_search.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Tuned Pipeline Accuracy: {accuracy:.2f}")

# Save the tuned pipeline
with open("pipeline_tuned.pkl", "wb") as f:
    pickle.dump(grid_search.best_estimator_, f)

print("Tuned pipeline saved as 'pipeline_tuned.pkl'")
```

Pipeline – sklearn with Hyper-Parameter Tuning

sklearn_gridsearch_pipeline.py

```
# Load the saved pipeline
with open("pipeline_tuned.pkl", "rb") as f:
    pipeline_tuned = pickle.load(f)

# Example input for prediction (same number of features as the training data: 4)
# Let's say this is a new Iris flower data point:
X_new = np.array([[5.1, 3.5, 1.4, 0.2]])

# Make prediction
prediction = pipeline_tuned.predict(X_new)

# Print result
print(f"Predicted class: {prediction[0]}")
```

Pipeline – sklearn with Hyper-Parameter Tuning

- clf__var_smoothing → double underscore __ separates step name (clf) and hyper-parameter name (var_smoothing) inside the pipeline.
- GridSearchCV automatically performs cross-validation to find the best hyperparameter.
- After fitting, grid_search.best_estimator_ is the final pipeline we can pickle and use in Flask.
- Always perform hyperparameter tuning on the pipeline, not on the raw model.
- Use GridSearchCV or RandomizedSearchCV.
- Save the best estimator from grid search (best_estimator_) for production / Flask deployment.

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd
from itertools import product



# 1. Dataset
df = pd.read_csv("Iris.csv")
df.drop(columns=['Id'], errors='ignore', inplace=True)

X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values   # already numeric

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# 2. TorchPipeline
class TorchPipeline:
    def __init__(self, hidden_size=16):
        self.scaler = StandardScaler()
        self.hidden_size = hidden_size
        self.model = nn.Sequential(
            nn.Linear(4, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 3)
        )

    # custom preprocessing
    def custom_input_preprocessing(self, X):
        X = X.copy()
        X[:, 0] = np.log1p(X[:, 0])   # log-transform on feature 0
        return X
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# training
    def fit(self, X_train, y_train, lr=0.01, epochs=100):
        X_proc = self.custom_input_preprocessing(X_train)
        X_scaled = self.scaler.fit_transform(X_proc)

        X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
        y_tensor = torch.tensor(y_train, dtype=torch.long)

        optimizer = optim.Adam(self.model.parameters(), lr=lr)
        loss_fn = nn.CrossEntropyLoss()

        for epoch in range(epochs):
            self.model.train()
            optimizer.zero_grad()
            outputs = self.model(X_tensor)
            loss = loss_fn(outputs, y_tensor)
            loss.backward()
            optimizer.step()
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# prediction (numeric labels)
    def predict(self, X):
        X_proc = self.custom_input_preprocessing(X)
        X_scaled = self.scaler.transform(X_proc)
        X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
        self.model.eval()
        with torch.no_grad():
            logits = self.model(X_tensor)
            preds = torch.argmax(logits, dim=1).numpy()
        return preds

    # saving
    def save(self, path):
        torch.save(self, path)  # save whole pipeline object

    # loading
    @staticmethod
    def load(path):
        return torch.load(path)
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# hyperparameter tuning
    @staticmethod
    def tune(X_train, y_train, X_val, y_val,
             hidden_sizes=[8, 16, 32],
             lrs=[0.01, 0.001],
             epochs=[50, 100]):
        best_acc = 0.0
        best_params = None
        best_pipeline = None

        for hidden_size, lr, n_epochs in product(hidden_sizes, lrs, epochs):
            pipeline = TorchPipeline(hidden_size=hidden_size)
            pipeline.fit(X_train, y_train, lr=lr, epochs=n_epochs)

            preds = pipeline.predict(X_val)
            acc = np.mean(preds == y_val)

            print(f"Hidden={hidden_size}, LR={lr}, Epochs={n_epochs} -> Acc={acc:.4f}")

            if acc > best_acc:
                best_acc = acc
                best_params = (hidden_size, lr, n_epochs)
                best_pipeline = pipeline

        print(f"\nBest Params: Hidden={best_params[0]}, LR={best_params[1]}, Epochs={best_params[2]} (Acc={best_acc:.4f})")
        return best_pipeline, best_params, best_acc #class over
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# 3. Hyperparameter tuning
# Create a validation split from train
X_subtrain, X_val, y_subtrain, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42
)

best_pipeline, best_params, best_acc = TorchPipeline.tune(
    X_subtrain, y_subtrain, X_val, y_val,
    hidden_sizes=[8, 16, 32],
    lrs=[0.01, 0.001],
    epochs=[50, 100]
)
```

Pipeline – PyTorch with Hyperparameter Tuning

train_pytorch_tuning.py

```
# 4. Final evaluation
preds = best_pipeline.predict(X_test)
acc = np.mean(preds == y_test)
print(f"\nFinal Test Accuracy (best model): {acc:.4f}")

best_pipeline.save("best_pipeline.pth")

# 5. Load + Predict
loaded_pipeline = TorchPipeline.load("best_pipeline.pth")
sample = X_test[0].reshape(1, -1)
print("Predicted:", loaded_pipeline.predict(sample)[0], "Actual:", y_test[0])
```

Disclaimer

- Content of this presentation is not original and it has been prepared from various sources for teaching purpose.